Projet— Dungeon as a Service

Ce projet a pour objectif de permettre la mise en application de l'ensemble des compétences travaillées dans le module Python Backend & FastAPI, sur une séance de 4 heures.

Il ne s'agit pas uniquement de réaliser une API fonctionnelle, mais de produire un projet complet permettant de démontrer la maîtrise des compétences attendues : conception d'une API REST, validation et persistance des données, programmation orientée objet, gestion des erreurs HTTP, programmation asynchrone, sécurisation des accès et documentation.

Le projet se réalise en groupe de 2 ou 3 et se découpe en quatre paliers progressifs. Chaque palier validé rapporte des points, même si le suivant n'est pas atteint.

L'objectif

L'objectif de ce projet est de concevoir le moteur d'un jeu de rôle rétro nommé Dungeon as a Service, développé par le studio indépendant Pixel Crypt. Le jeu se joue uniquement depuis un terminal : tout le moteur de jeu est une API, le terminal n'étant qu'un client qui lui envoie des requêtes.

Le joueur crée un héros, explore un donjon maudit salle par salle, affronte les monstres qui le hantent et tente d'atteindre la salle du trône pour vaincre la Liche.

L'API devra permettre :

- Aux joueurs de créer et consulter leurs héros ;
- De se déplacer dans le donjon et d'en découvrir les salles ;
- De combattre les monstres et de se reposer ;
- De sécuriser les comptes joueurs afin que chacun ne contrôle que ses propres héros.

La carte du donjon (fichier JSON) ainsi qu'un squelette de projet vous sont fournis sur Moodle dans l'archive `dungeon-api-starter.zip`.

Mise en place

La base de données, la lecture de la carte et le serveur sont déjà configurés dans le squelette. La mise en place ne doit pas dépasser 15 minutes.

Les étapes attendues sont :

- Un membre du groupe décompresse le squelette, crée un dépôt Git et y invite les autres membres ;

© 2026 Enzo Giardinelli. Tous droits réservés

- Créer un environnement virtuel : `python -m venv .venv`, puis l'activer (Windows : `.venv\Scripts\Activate.ps1`, macOS / Linux : `source .venv/bin/activate`) ;
- Installer les dépendances : `pip install -r requirements.txt` ;
- Lancer le serveur depuis la racine du projet : `fastapi dev app/main.py` ;
- Ouvrir http://127.0.0.1:8000/docs et exécuter la route `GET /health`.

Si la route `/health` ne répond pas au bout de 15 minutes, prévenez le mentor sur Teams sans attendre.

Le squelette contient les éléments suivants :

Élément	Rôle	À modifier
app/main.py	Point d'entrée, branchement des routers	Oui
app/database.py	Connexion SQLite, get_session()	Non
app/services/dungeon_map.py	Lecture de la carte : get_room(), get_monster_of_room(), START_ROOM, FINAL_ROOM	Non
data/dungeon_map.json	Carte du donjon (8 salles, 5 monstres)	Non
app/models, schemas, domain, services, routes	Dossiers à compléter, chacun avec une consigne en commentaire	Oui
client/play.py	Squelette du client terminal	Bonus

Fonctionnalités

Les fonctionnalités obligatoires du projet sont divisées en quatre paliers à réaliser dans l'ordre :

- Palier 1 – Le héros (environ 1h) ;
- Palier 2 – Le donjon (environ 1h) ;
- Palier 3 – Le combat (environ 45 minutes) ;
- Palier 4 – Le verrou (environ 45 minutes).

Chaque palier se termine par une validation à effectuer dans Swagger. Ne passez au palier suivant qu'une fois cette validation réussie : un palier terminé proprement vaut mieux que deux paliers à moitié faits.

Fonctionnalités du palier 1 – Le héros

FT-1 Création d'un héros

Vous devrez implémenter une route `POST /heroes` permettant de créer un héros. La route renvoie le héros créé avec le code 201. Le héros apparaît dans la salle de départ (START_ROOM) avec ses points de vie au maximum.

© 2026 Enzo Giardinelli. Tous droits réservés

Chaque héros devra être défini par les attributs suivants :

- nom
- classe
- force
- agilité
- intelligence

Exemple de requête attendue :

```
{ "nom": "Aragorn", "classe": "guerrier", "force": 6, "agilite": 2, "intelligence": 2 }
```

FT-2 Validation de la création

La validation des données devra être réalisée avec Pydantic, et non avec des conditions dans la route.

Toute donnée invalide renverra une erreur 422.

Les contraintes liées à la création d'un héros sont les suivantes :

- Le nom doit être une chaîne de caractères d'une longueur comprise entre 3 et 20 caractères ;
- La classe doit être l'une des valeurs suivantes : guerrier, mage ou voleur ;
- Chaque statistique doit être un entier compris entre 1 et 8 ;
- La somme des trois statistiques doit valoir exactement 10.

FT-3 Consultation des héros

Vous devrez implémenter un système permettant de consulter les héros existants.

Les opérations attendues sont :

- Récupérer la liste des héros : `GET /heroes` ;
- Consulter un héros à partir de son identifiant : `GET /heroes/{id}` ;
- Renvoyer une erreur 404 au format imposé (voir les contraintes techniques) lorsque l'identifiant est inconnu.

Validation du palier 1 : dans Swagger, créez un guerrier valide (201), tentez une création dont la somme des statistiques vaut 15 (422), puis demandez le héros 9999 (404 structurée).

Fonctionnalités du palier 2 – Le donjon

FT-4 Persistance des héros

Vous devrez stocker les héros en base de données SQLite à l'aide de SQLModel. Les héros devront être conservés après un redémarrage du serveur.

© 2026 Enzo Giardinelli. Tous droits réservés

FT-5 Description de la salle

Vous devrez implémenter une route `GET /heroes/{id}/room` permettant de décrire la salle dans laquelle se trouve le héros.

La description devra contenir au minimum les informations suivantes :

- identifiant de la salle
- nom
- description
- liste des sorties disponibles
- monstre présent, si un combat est en cours

FT-6 Déplacement

Vous devrez implémenter une route `POST /heroes/{id}/move` permettant de déplacer le héros. Elle reçoit une direction, par exemple `{"direction": "nord"}`, et renvoie la nouvelle salle.

Les contraintes liées au déplacement sont les suivantes :

- La direction doit être l'une des valeurs suivantes : nord, sud, est ou ouest. Toute autre valeur renverra une erreur 422 ;
- Si la salle ne possède pas de sortie dans cette direction, une erreur 400 avec le code NO_EXIT devra être renvoyée ;
- Entrer dans une salle hantée démarre un combat : le monstre apparaît avec ses points de vie au maximum.

Validation du palier 2 : depuis l'entrée, déplacez-vous vers le nord (hall), tentez une direction sans sortie (400), redémarrez le serveur et vérifiez que le héros se trouve toujours dans le hall.

Fonctionnalités du palier 3 – Le combat

FT-7 Attaque

Vous devrez implémenter une route `POST /heroes/{id}/attack` permettant de dérouler un tour de combat. Le héros frappe en premier. Si le monstre survit, il riposte.

La réponse devra détailler le tour de combat :

```
{ "degats_infliges": 14, "critique": false, "degats_subis": 3, "pv_heros": 29,
  "pv_monstre": 1, "issue": "en_cours", "or_gagne": 0, "victoire": false }
```

Le champ issue peut prendre les valeurs suivantes : en_cours, monstre_vaincu ou heros_mort.

© 2026 Enzo Giardinelli. Tous droits réservés

Le calcul des dégâts devra obligatoirement passer par les classes métier (voir les entités métiers), et non par des conditions sur la classe du héros dans le service.

FT-8 Gestion des états du héros

Vous devrez empêcher les actions incohérentes avec l'état du héros. Ces actions renverront une erreur 409 avec un code explicite.

Les cas attendus sont :

- Se déplacer pendant un combat : IN_COMBAT ;
- Attaquer alors qu'aucun monstre n'est présent : NO_MONSTER ;
- Effectuer une action (déplacement, attaque, repos) avec un héros mort : HERO_DEAD.

Un héros mort doit rester consultable avec les routes de consultation.

FT-9 Repos

Vous devrez implémenter une route asynchrone `POST /heroes/{id}/rest` (`async def`). Le héros se repose pendant 3 secondes (`await asyncio.sleep(3)`) puis récupère l'ensemble de ses points de vie.

Le repos est interdit pendant un combat (409, IN_COMBAT). Pendant les 3 secondes de repos, le serveur doit continuer à répondre aux autres requêtes.

Validation du palier 3 : battez le gobelin de la bibliothèque, vérifiez que l'or du héros a augmenté, reposez-vous, puis tentez de quitter une salle hantée sans combattre (409).

Fonctionnalités du palier 4 – Le verrou

FT-10 Authentification

Vous devrez implémenter un système d'authentification permettant aux joueurs de s'identifier afin d'accéder à leurs héros.

Les opérations attendues sont :

- Créer un compte : `POST /auth/register` ;
- Se connecter : `POST /auth/login`, qui renvoie un token JWT.

Les contraintes liées à l'authentification sont les suivantes :

- Le mot de passe doit être stocké haché, jamais en clair ;
- L'utilisation de tokens JWT est attendue pour la gestion des sessions ;
- La route de connexion devra utiliser OAuth2PasswordRequestForm afin que le bouton Authorize de Swagger soit fonctionnel.

© 2026 Enzo Giardinelli. Tous droits réservés

FT-11 Protection des héros

Toutes les routes /heroes devront exiger un token valide. Chaque héros est rattaché au joueur qui l'a créé.

Les contraintes attendues sont :

- Une requête sans token ou avec un token invalide renverra une erreur 401 ;
- Agir sur le héros d'un autre joueur renverra une erreur 403 avec le code NOT_YOUR_HERO ;
- La route `GET /heroes` ne listera que les héros du joueur connecté.

Validation du palier 4 : créez deux comptes. Avec le premier, créez un héros. Avec le second, tentez de le déplacer (403). Sans token, `GET /heroes` doit renvoyer une erreur 401.

Entités métiers

Les règles du jeu devront être implémentées en programmation orientée objet dans le dossier app/domain/, sans aucune dépendance à FastAPI ni à la base de données. Le service construit l'objet métier à partir du héros stocké, puis lui délègue le calcul.

Classe abstraite Heros

La classe Heros hérite de ABC et porte les attributs force, agilité et intelligence. Elle expose une propriété pv_max égale à 20 + force × 2.

Elle déclare les méthodes suivantes :

- `calculer_degats()` : méthode abstraite, renvoie les dégâts infligés et indique si le coup est critique ;
- `subir_degats(attaque_monstre)` : renvoie les dégâts réellement subis. Par défaut, il s'agit de l'attaque du monstre.

Les sous-classes attendues sont les suivantes :

Classe	calculer_degats()	subir_degats()	Exemple (6 dans la statistique principale)
Guerrier	2 + force × 2	Armure : attaque − 1, minimum 1	14 dégâts, 32 PV
Mage	1 + intelligence × 3	Héritée	19 dégâts, 24 PV
Voleur	2 + agilité × 2, coup critique (×2) une fois sur trois	Héritée	14 dégâts (28 en critique), 24 PV

Héros

Le héros stocké en base devra contenir au minimum les informations suivantes :

© 2026 Enzo Giardinelli. Tous droits réservés

Champ	Type	Remarque
id	int	Clé primaire
nom, classe	str	
force, agilite, intelligence	int	
pv, pv_max	int	Le héros est mort lorsque pv vaut 0
salle_actuelle	str	Identifiant d'une salle de la carte
or_	int	0 au départ
monstre_pv	int ou null	null : aucun combat en cours
victoire	bool	Vrai lorsque la Liche est vaincue
owner_id	int	Palier 4 : clé étrangère vers le joueur

Les champs monstre_pv et owner_id ne doivent pas apparaître tels quels dans les réponses de l'API. Exposez plutôt les informations en_combat et est_vivant dans votre schéma de sortie.

Joueur

Chaque joueur devra être défini par les attributs suivants :

- identifiant
- nom d'utilisateur (unique)
- mot de passe haché

Règles du donjon

Les règles du donjon sont les suivantes :

- La carte compte 8 salles et 5 monstres : Rat géant (8 PV, attaque 2, 5 or), Gobelin (12 PV, attaque 3, 10 or), Squelette (15 PV, attaque 4, 15 or), Ogre (30 PV, attaque 6, 40 or) et La Liche (50 PV, attaque 7, 100 or) ;
- Le donjon est maudit : un monstre réapparaît chaque fois que l'on entre dans sa salle ;
- Vaincre un monstre rapporte son or. Vaincre la Liche dans la salle finale (FINAL_ROOM) fait passer victoire à vrai ;
- Avec un repos avant la salle du trône, chaque classe est en mesure de vaincre la Liche.

Fonctionnalités bonus

Les fonctionnalités bonus peuvent être réalisées dans l'ordre de votre choix. Elles rapportent au maximum 2 points et peuvent compenser des points perdus ailleurs, dans la limite de 20/20.

© 2026 Enzo Giardinelli. Tous droits réservés

B-1 Client terminal

Vous pourrez compléter le fichier client/play.py avec argparse et requests afin de jouer depuis le terminal. Une sous-commande sera attendue par action : register, login, create, look, move, attack et rest. Le token devra être sauvegardé dans un fichier .token.

B-2 Classement

Vous pourrez implémenter une route publique `GET /leaderboard` renvoyant les 10 meilleurs héros : les vainqueurs d'abord, puis par quantité d'or.

B-3 Tests

Vous pourrez écrire au minimum 5 tests pytest avec TestClient, couvrant un cas nominal et des cas d'erreur (422, 400, 409, 403).

B-4 PostgreSQL

Vous pourrez fournir un fichier docker-compose.yml lançant PostgreSQL. L'API devra s'y connecter via la variable d'environnement DATABASE_URL, sans modification du code.

B-5 Journalisation des combats

Vous pourrez utiliser le module logging afin de tracer chaque déplacement et chaque tour de combat dans la console.

Les contraintes techniques

Les technologies obligatoires du projet sont les suivantes :

Domaine	Technologies imposées
Backend	Python 3.10 ou supérieur, FastAPI, Pydantic
Base de données	SQLite (PostgreSQL en bonus)
ORM / accès aux données	SQLModel
Authentification	JWT (PyJWT), hachage avec pwdlib
Programmation asynchrone	async / await, asyncio
Tests (bonus)	pytest
Client terminal (bonus)	argparse, requests
Versionnement	Git
Documentation API	OpenAPI générée par FastAPI

© 2026 Enzo Giardinelli. Tous droits réservés

Les contraintes d'architecture et de code sont les suivantes :

- Les routes ne contiennent que la couche HTTP. La logique est placée dans services/, les règles du jeu dans domain/, les tables dans models/ et les schémas d'entrée et de sortie dans schemas/ ;
- Les fichiers data/dungeon_map.json, app/database.py et app/services/dungeon_map.py ne doivent pas être modifiés ;
- Chaque route doit apparaître dans Swagger avec un response_model et un tag ;
- Les codes de statut attendus sont 201 à la création, 204 à la suppression (si elle est implémentée) et 200 dans les autres cas ;
- La clé secrète JWT doit être lue depuis une variable d'environnement, avec une valeur par défaut pour le développement ;
- Un dépôt Git par groupe est attendu, avec des commits réguliers de chaque membre. Un projet comportant un seul commit en fin de séance sera pénalisé.

Toutes les erreurs métier (400, 403, 404, 409) devront respecter le format suivant :

```
{ "detail": { "error": "NO_EXIT", "message": "Il n'y a pas de passage vers le
est." } }
```

Ce format s'obtient avec `raise HTTPException(status_code=400, detail={"error": "...", "message": "..."})`. Les erreurs 422 restent celles générées automatiquement par FastAPI.

Livrables

Le dépôt sur Moodle se fait à la fin de la séance, à l'heure indiquée par le mentor. Un seul dépôt est attendu par groupe.

Les éléments attendus sont :

- Une archive .zip du projet, sans les dossiers .venv/ et sans le fichier dungeon.db ;
- Le fichier README complété (section « Notre rendu ») : membres du groupe, lien du dépôt Git, palier atteint, bonus réalisés et éléments non fonctionnels ;
- Le fichier openapi.json exporté depuis http://127.0.0.1:8000/openapi.json et placé à la racine du projet.

Un palier est considéré comme atteint lorsque sa validation est réussie. Indiquez honnêtement ce qui ne fonctionne pas : un bug signalé et expliqué sera moins pénalisé qu'un bug dissimulé.

© 2026 Enzo Giardinelli. Tous droits réservés

Grille d'évaluation

La note est établie sur 20 points, auxquels peuvent s'ajouter jusqu'à 2 points de bonus, dans la limite de 20/20.

Critères	Détails	Barème
Palier 1 – Le héros		**5**
FT-1 Création	POST /heroes renvoie 201, héros en salle de départ avec PV au maximum	1,5
FT-2 Validation	Les 4 contraintes appliquées par Pydantic ; erreur 422 sur chaque cas invalide	2
FT-3 Consultation	Liste et détail fonctionnels ; erreur 404 au format imposé	1,5
Palier 2 – Le donjon		**4**
FT-4 Persistance	Table SQLModel, données conservées après redémarrage	1
FT-5 Salle	Description complète : nom, description, sorties, monstre	1
FT-6 Déplacement	Déplacement correct ; erreurs 422 et 400 NO_EXIT ; combat déclenché en salle hantée	2
Palier 3 – Le combat		**5**
Programmation orientée objet	Classe abstraite Heros et trois sous-classes ; calculer_degats() redéfinie, subir_degats() redéfinie pour le guerrier ; aucune condition sur la classe dans le service	2
FT-7 Attaque	Tour de combat correct (ordre, riposte, or, victoire finale) ; réponse détaillée	1
FT-8 États du héros	Erreurs 409 IN_COMBAT, NO_MONSTER, HERO_DEAD ; héros mort toujours consultable	1
FT-9 Repos	Route async def avec await asyncio.sleep ; PV restaurés ; erreur 409 en combat	1
Palier 4 – Le verrou		**3**
FT-10 Authentification	Mot de passe haché, connexion JWT, bouton Authorize fonctionnel	1,5
FT-11 Protection	Erreurs 401 et 403 NOT_YOUR_HERO, liste filtrée par joueur	1,5
Qualité et rendu		**3**
Architecture	Séparation routes, services, domaine, modèles et schémas respectée	1
Rigueur de l'API	Format d'erreur respecté partout ; response_model et tags sur chaque route	1
Rendu	Commits réguliers de chaque membre, README complété, openapi.json fourni	1
Total		**20**
Bonus (2 points maximum)	B-1 client terminal : 1 ; B-2 classement : 0,5 ; B-3 tests : 0,5 ; B-4 PostgreSQL : 0,5 ; B-5 journalisation : 0,5	+2

Il est important de noter qu'une explication de votre solution pourra vous être demandée. L'usage de l'IA est autorisé, mais chaque membre du groupe doit être capable d'expliquer n'importe quelle partie du code rendu.

© 2026 Enzo Giardinelli. Tous droits réservés